
Oliver Krauss

Basics of Neural Networks and Deep Learning

AIN2IL | MTD.ba SS2026

Hagenberg 04.05.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

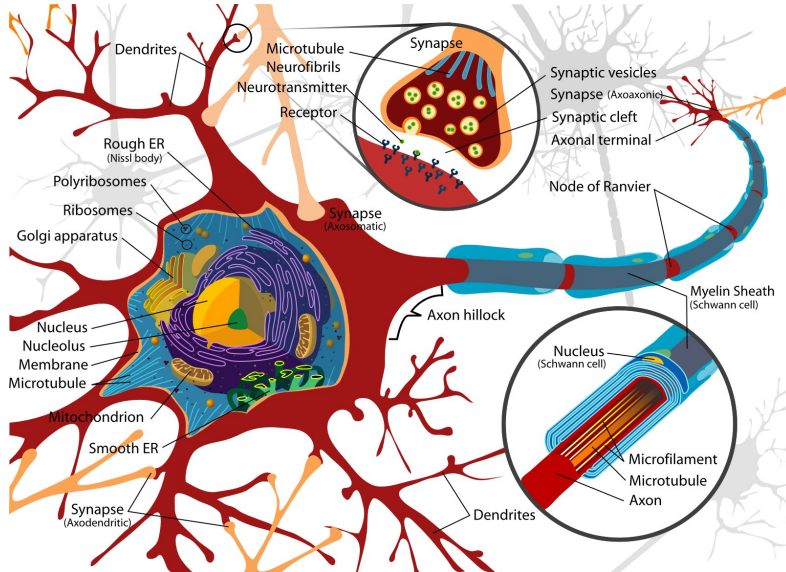
Introduction

- The most powerful and most versatile *learning machine* is still the human brain.
- Starting in the 1940ies, ideas for creating *intelligent* systems by mimicking the function of nerve/brain cells have been developed.
- An *artificial neural network* is a parallel processing system with small computing units (*neurons*) that work similarly to nerve/brain cells.

Neurophysiological Background I

- The inside of every neuron (nerve or brain cell) carries a certain electric charge.
- Electric charge of connected neurons may raise or lower this charge (by means of transmission of ions through the synaptic interface).
- As soon as the charge reaches a certain threshold, an electric impulse is transmitted through the cell's axon to the neighboring cells.
- In the synaptic interfaces, chemicals called neurotransmitters control the strength to which an impulse is transmitted from one cell to another.

Neurophysiological Background II



Hebb's Learning Rule

- Postulated by Donald Hebb in 1949; it is the first ever learning rule that describes how the neuronal activities influence the connection between neurons, i.e., the adaptation of synapses.
- Simplistic formulation: "Cells that fire together wire together."
- In other words, two physically connected cells strengthen their connection if they are firing together.

Feed-Forward Neural Networks

- We restrict to *feed-forward neural networks*, i.e. simple static input-output systems without any feedback loops between neurons and without any system dynamics over time.
- Within this class, we consider perceptrons and multi-layer perceptrons (along with the backpropagation algorithm).
- Finally, we will also highlight *deep learning*, i.e. different strategies for training networks with many layers (which mostly have large numbers of neurons too).

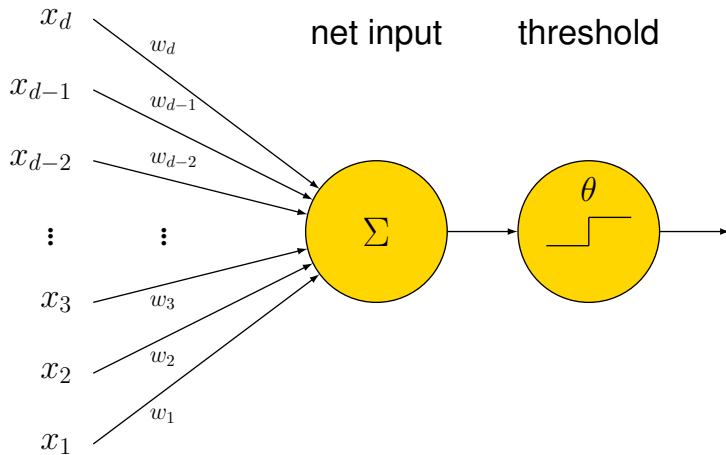
Perceptrons I

- A perceptron is a simple linear threshold unit:

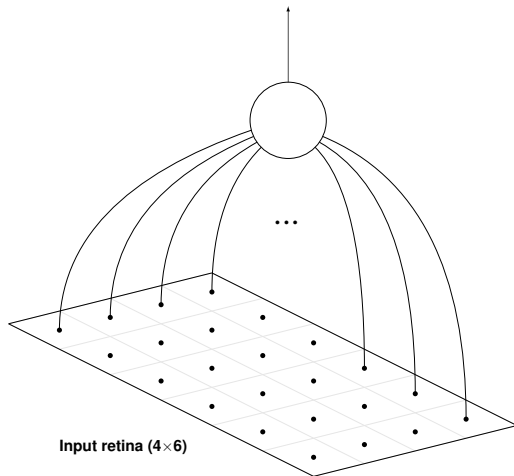
$$g(\vec{x}; \vec{w}, \theta) = \begin{cases} 1 & \text{if } \sum_{j=1}^d w_j \cdot x_j > \theta \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

- In analogy to the biological model, the inputs x_j correspond to the charges received from connected cells through the dendrites, the weights w_j correspond to the properties of the synaptic interface, and the output corresponds to the impulse that is sent through the axon as soon as the charge exceeds the threshold θ .
- Though it seems to be a (simplistic) model of a neuron, a perceptron is nothing else but a simple linear classifier.

Perceptrons II



Perceptrons: The Retina Metaphor



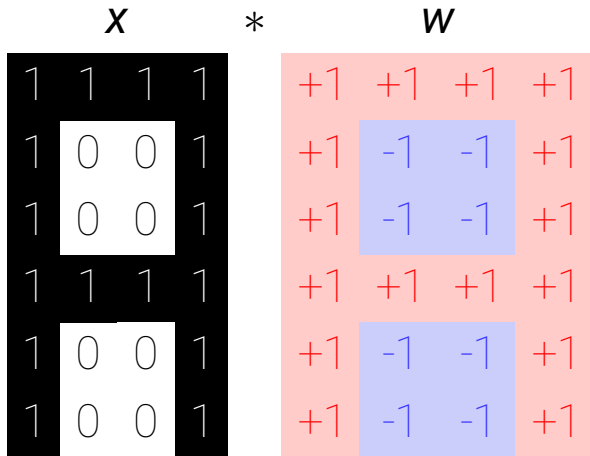
Example: 'A' Detector

w

+1	+1	+1	+1
+1	-1	-1	+1
+1	-1	-1	+1
+1	+1	+1	+1
+1	-1	-1	+1
+1	-1	-1	+1

$$\theta = 15.5$$

Example: 'A' Detector



$$\theta = 15.5$$

Example: 'A' Detector

$$\begin{array}{c} \mathbf{X} \end{array} * \begin{array}{c} \mathbf{W} \end{array} \rightarrow \Sigma$$

The input matrix $\mathbf{X}$ is a 6x4 grid with a black background and white '1's forming the shape of the letter 'A'. The weight matrix $\mathbf{W}$ is a 6x4 grid with a light red background and blue '-1's in the center, forming the shape of the letter 'A'. The output is $\theta = 15.5$.

$\mathbf{X}$	$\mathbf{W}$
1 1 1 1	+1 +1 +1 +1
1 0 0 1	+1 -1 -1 +1
1 0 0 1	+1 -1 -1 +1
1 1 1 1	+1 +1 +1 +1
1 0 0 1	+1 -1 -1 +1
1 0 0 1	+1 -1 -1 +1

Example: 'A' Detector

$$\begin{array}{c} \mathbf{X} \end{array} * \begin{array}{c} \mathbf{W} \end{array} \rightarrow \begin{array}{c} \Sigma \\ \downarrow \\ 16 \end{array}$$

$\theta = 15.5$

The diagram illustrates the dot product of an input matrix $\mathbf{X}$ and a weight matrix $\mathbf{W}$ to detect the digit 'A'. The input matrix $\mathbf{X}$ is a 6x4 grid where the background is black and the digit 'A' is represented by white cells with black outlines. The weight matrix $\mathbf{W}$ is a 6x4 grid where the background is light red and the digit 'A' is represented by a blue-outlined shape. The weights are +1 for the background and -1 for the digit 'A'. The dot product is calculated as the sum of the element-wise products, resulting in 16.

$\mathbf{X}$	$\mathbf{W}$
1 1 1 1	+1 +1 +1 +1
1 0 0 1	+1 -1 -1 +1
1 0 0 1	+1 -1 -1 +1
1 1 1 1	+1 +1 +1 +1
1 0 0 1	+1 -1 -1 +1
1 0 0 1	+1 -1 -1 +1

Example: 'A' Detector

$$\begin{array}{c} \mathbf{X} \end{array} * \begin{array}{c} \mathbf{W} \end{array} \rightarrow \begin{array}{c} \Sigma \\ \downarrow \\ 16 \\ > \\ \theta = 15.5 \end{array}$$

The diagram illustrates the dot product of an input matrix $\mathbf{X}$ and a weight matrix $\mathbf{W}$ to detect the digit 'A'.

Input Matrix $\mathbf{X}$: A 6x4 matrix where the background is black (1) and the digit 'A' is white (0).

1	1	1	1
1	0	0	1
1	0	0	1
1	1	1	1
1	0	0	1
1	0	0	1

Weight Matrix $\mathbf{W}$: A 6x4 matrix with red (+1) and blue (-1) values.

+1	+1	+1	+1
+1	-1	-1	+1
+1	-1	-1	+1
+1	+1	+1	+1
+1	-1	-1	+1
+1	-1	-1	+1

The result of the dot product is $\Sigma = 16$, which is greater than the threshold $\theta = 15.5$.

Example: 'A' Detector

$$\begin{array}{c} \mathbf{X} \\ \begin{array}{|c|c|c|c|} \hline 1 & 1 & 1 & 1 \\ \hline 1 & 0 & 0 & 1 \\ \hline 1 & 0 & 0 & 1 \\ \hline 1 & 1 & 1 & 1 \\ \hline 1 & 0 & 0 & 1 \\ \hline 1 & 0 & 0 & 1 \\ \hline \end{array} \end{array} * \begin{array}{c} \mathbf{W} \\ \begin{array}{|c|c|c|c|} \hline +1 & +1 & +1 & +1 \\ \hline +1 & -1 & -1 & +1 \\ \hline +1 & -1 & -1 & +1 \\ \hline +1 & +1 & +1 & +1 \\ \hline +1 & -1 & -1 & +1 \\ \hline +1 & -1 & -1 & +1 \\ \hline \end{array} \end{array} \rightarrow \begin{array}{c} \Sigma \\ \downarrow \\ 16 \\ > \\ \theta = 15.5 \\ \downarrow \\ \mathbf{1} \end{array}$$

Example: 'A' Detector

$$\begin{array}{c} \mathbf{X} \end{array} * \begin{array}{c} \mathbf{W} \end{array} \rightarrow \begin{array}{c} \Sigma \\ \downarrow \\ 15 \end{array}$$

$\theta = 15.5$

The diagram illustrates the dot product of an input matrix $\mathbf{X}$ and a weight matrix $\mathbf{W}$ to detect the digit 'A'. The input matrix $\mathbf{X}$ is a 7x4 grid where the digit 'A' is represented by black cells (value 1) and the background by white cells (value 0). The weight matrix $\mathbf{W}$ is a 7x4 grid where the weights for detecting 'A' are shown: red for positive weights (+1) and blue for negative weights (-1). The dot product is calculated by multiplying corresponding elements of $\mathbf{X}$ and $\mathbf{W}$ and then summing the results. The sum is 15, which is then used to calculate the threshold $\theta = 15.5$.

$\mathbf{X}$	$\mathbf{W}$
1 1 1 1	+1 +1 +1 +1
1 0 0 1	+1 -1 -1 +1
1 0 0 1	+1 -1 -1 +1
1 1 0 1	+1 +1 +1 +1
1 0 0 1	+1 -1 -1 +1
1 0 0 1	+1 -1 -1 +1

Example: 'A' Detector

$$\begin{array}{c} \mathbf{X} \\ \begin{array}{|c|c|c|c|} \hline 1 & 1 & 1 & 1 \\ \hline 1 & 0 & 0 & 1 \\ \hline 1 & 0 & 0 & 1 \\ \hline 1 & 1 & 0 & 1 \\ \hline 1 & 0 & 0 & 1 \\ \hline 1 & 0 & 0 & 1 \\ \hline \end{array} \end{array} * \begin{array}{c} \mathbf{W} \\ \begin{array}{|c|c|c|c|} \hline +1 & +1 & +1 & +1 \\ \hline +1 & -1 & -1 & +1 \\ \hline +1 & -1 & -1 & +1 \\ \hline +1 & +1 & +1 & +1 \\ \hline +1 & -1 & -1 & +1 \\ \hline +1 & -1 & -1 & +1 \\ \hline \end{array} \end{array} \rightarrow \begin{array}{c} \Sigma \\ \downarrow \\ 15 \\ < \\ \theta = 15.5 \end{array}$$

Example: 'A' Detector

$$\begin{array}{c} \mathbf{X} \end{array} * \begin{array}{c} \mathbf{W} \end{array} \rightarrow \begin{array}{c} \Sigma \\ \downarrow \\ 15 \\ < \\ \theta = 15.5 \\ \downarrow \\ \mathbf{0} \end{array}$$

The diagram illustrates the dot product of an input vector $\mathbf{X}$ and a weight vector $\mathbf{W}$ for an 'A' detector. The input vector $\mathbf{X}$ is a 6x4 grid where black cells are 1 and white cells are 0. The weight vector $\mathbf{W}$ is a 6x4 grid where red cells are +1, blue cells are -1, and white cells are 0. The dot product is calculated as the sum of the element-wise products, resulting in 15. This value is compared to the threshold $\theta = 15.5$. Since 15 is less than 15.5, the output is 0.

Example: 'A' Detector

$$\begin{array}{c} \mathbf{X} \end{array} * \begin{array}{c} \mathbf{W} \end{array} \rightarrow \begin{array}{c} \Sigma \\ \downarrow \\ 15 \end{array}$$

$\theta = 15.5$

The diagram illustrates the dot product of an input matrix $\mathbf{X}$ and a weight matrix $\mathbf{W}$ to detect the digit 'A'. The input matrix $\mathbf{X}$ is a 6x4 grid where black cells are 1 and white cells are 0. The weight matrix $\mathbf{W}$ is a 6x4 grid where red cells are +1, blue cells are -1, and white cells are 0. The dot product is calculated by multiplying corresponding elements and summing the results.

$\mathbf{X}$	$\mathbf{W}$
1 1 1 1	+1 +1 +1 +1
1 0 0 1	+1 -1 -1 +1
1 0 0 1	+1 -1 -1 +1
1 1 1 1	+1 +1 +1 +1
1 0 0 1	+1 -1 -1 +1
1 0 1 1	+1 -1 -1 +1

Example: 'A' Detector

$$\begin{array}{c} \mathbf{X} \end{array} * \begin{array}{c} \mathbf{W} \end{array} \rightarrow \begin{array}{c} \Sigma \\ \downarrow \\ 15 \\ < \\ \theta = 15.5 \end{array}$$

The diagram illustrates the dot product of an input matrix $\mathbf{X}$ and a weight matrix $\mathbf{W}$ to detect the digit 'A'.

Input Matrix $\mathbf{X}$ (6x4):

1	1	1	1
1	0	0	1
1	0	0	1
1	1	1	1
1	0	0	1
1	0	1	1

Weight Matrix $\mathbf{W}$ (6x4):

+1	+1	+1	+1
+1	-1	-1	+1
+1	-1	-1	+1
+1	+1	+1	+1
+1	-1	-1	+1
+1	-1	-1	+1

The result of the dot product is a sum of 15, which is less than the threshold $\theta = 15.5$.

Example: 'A' Detector

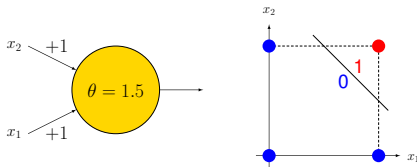
$$\begin{array}{c} \mathbf{X} \end{array} * \begin{array}{c} \mathbf{W} \end{array} \rightarrow \begin{array}{c} \Sigma \\ \downarrow \\ 15 \\ < \\ \theta = 15.5 \\ \downarrow \\ \mathbf{0} \end{array}$$

The diagram illustrates the dot product of an input vector $\mathbf{X}$ and a weight vector $\mathbf{W}$ for an 'A' detector. The input vector $\mathbf{X}$ is a 6x4 grid of values (1 or 0) on a black background. The weight vector $\mathbf{W}$ is a 6x4 grid of values (+1 or -1) on a light red background, with a central 2x2 blue region containing -1 values. The result of the dot product is a scalar value Σ , which is 15. This value is compared to a threshold $\theta = 15.5$. Since 15 is less than 15.5, the final output is 0.

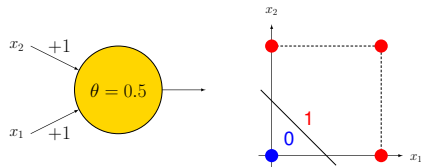
Example: Logical Gates / Clauses

Any logical clause (disjunctive or conjunctive) can be modeled with a perceptron. That particularly includes standard logical gates.

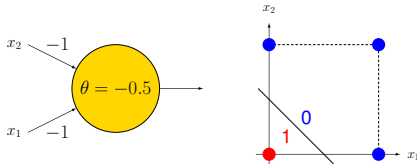
$x_1 \wedge x_2$ (AND):



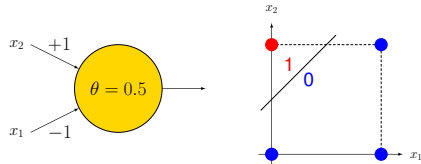
$x_1 \vee x_2$ (OR):



$\neg(x_1 \vee x_2) = \neg x_1 \wedge \neg x_2$ (NOR):



$\neg x_1 \wedge x_2$:



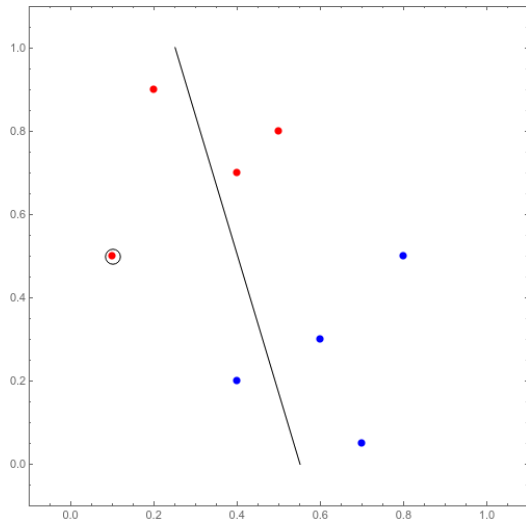
The Perceptron Learning Algorithm

1. Given: data set $\mathbf{Z} = \{(\vec{x}^i, y^i) \mid i = 1, \dots, l\}$, where $\vec{x}_i \in \mathbb{R}^d$, $y^i \in \{0, 1\}$; learning rate η ; initial weight vector $\vec{w}$, initial threshold θ ;
2. For $k = 1, \dots, l$ do:
 - If $g(\vec{x}^k; \vec{w}, \theta) = 0$ and $y^k = 1$
 - $\mathbf{w} := \mathbf{w} + \eta \cdot \mathbf{x}^k$
 - $\theta := \theta - \eta$
 - Else if $g(\vec{x}^k; \vec{w}, \theta) = 1$ and $y^k = 0$
 - $\mathbf{w} := \mathbf{w} - \eta \cdot \mathbf{x}^k$
 - $\theta := \theta + \eta$
3. Return to 2. if stopping condition not fulfilled
4. Output: vector of weights $\mathbf{w} \in \mathbb{R}^d$, threshold θ

The Perceptron Learning Algorithm Reformulated

1. Given: data set $\mathbf{Z} = \{(\vec{x}^i, y^i) \mid i = 1, \dots, l\}$, where $\vec{x}_i \in \mathbb{R}^d$, $y^i \in \{0, 1\}$; learning rate η ; initial weight vector $\vec{w}$, initial threshold θ ;
2. For $k = 1, \dots, l$ do:
 - If $g(\vec{x}^k; \vec{w}, \theta) \neq y^k$:
 - $\delta = g(\vec{x}^k; \vec{w}, \theta) - y^k$
 - $\mathbf{w} := \mathbf{w} - \eta \cdot \delta \cdot \mathbf{x}^k$
 - $\theta := \theta + \eta \cdot \delta$
3. Return to 2. if stopping condition not fulfilled
4. Output: vector of weights $\mathbf{w} \in \mathbb{R}^d$, threshold θ

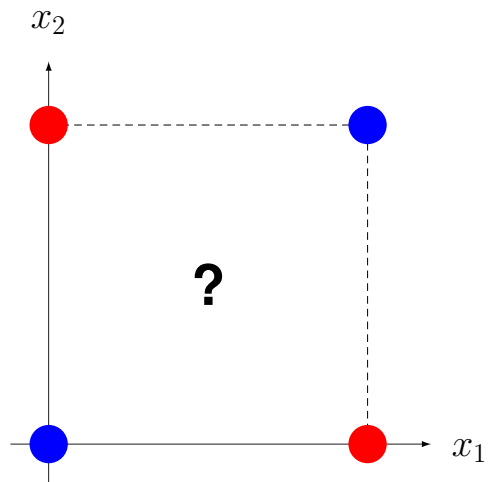
The Perceptron Learning Algorithm Example:



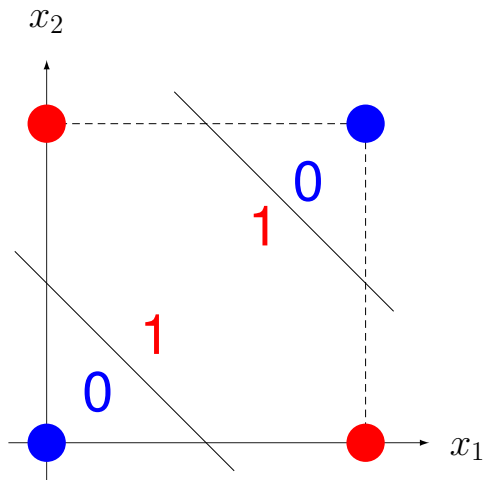
Perceptrons and Linear Separability

- In case that the data set $\mathbf{Z}$ is linearly separable in $\mathbb{R}^d$, the perceptron learning algorithm terminates and finally solves the learning task.
- The final solution is not unique and the learning algorithm just gives one arbitrary solution (depending on initial weights).
- Obviously, perceptrons cannot solve classification tasks that are not linearly separable, e.g. like the simple XOR problem. In such a case, the perceptron learning algorithm not even terminates.

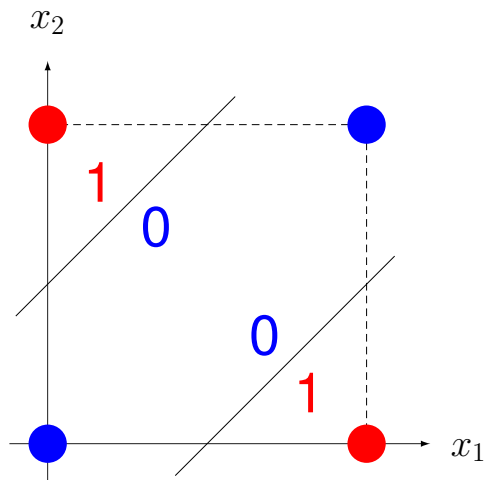
The XOR Problem



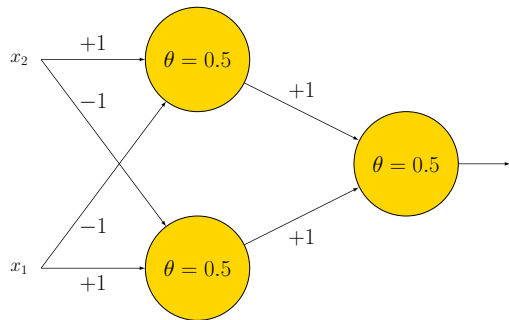
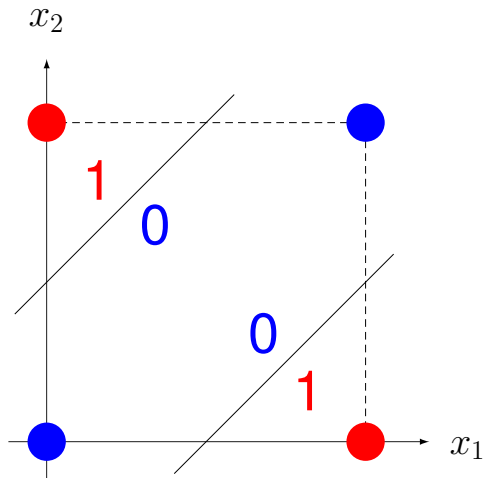
The XOR Problem



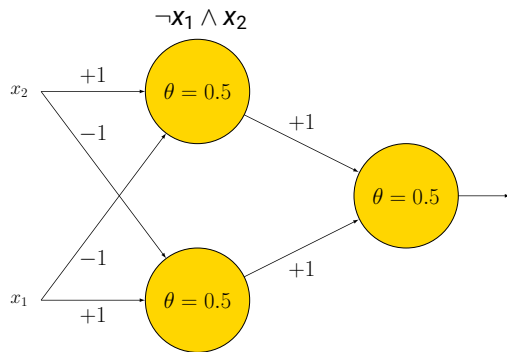
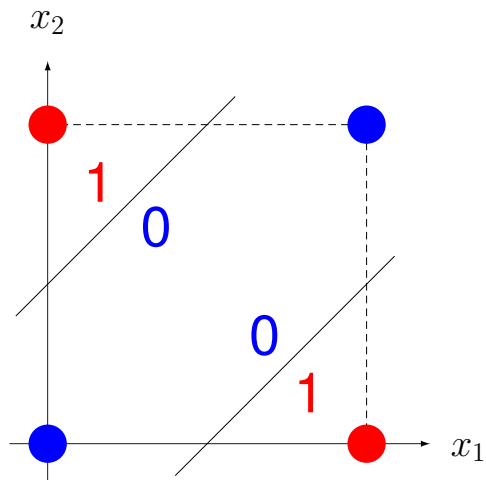
The XOR Problem



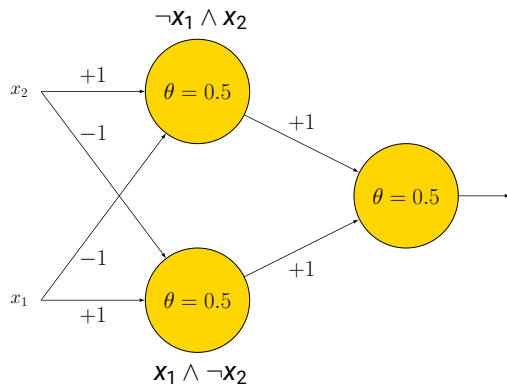
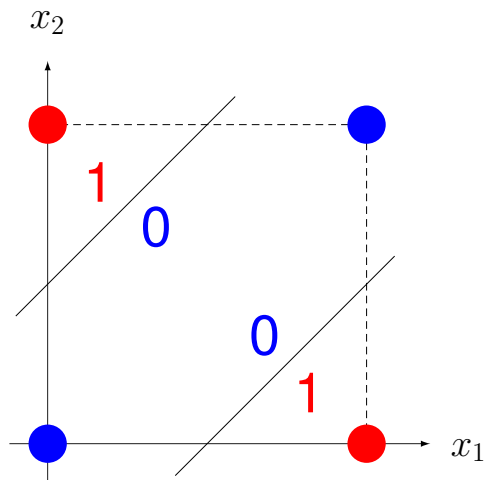
The XOR Problem



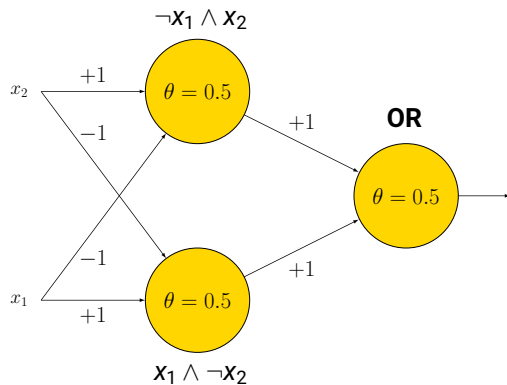
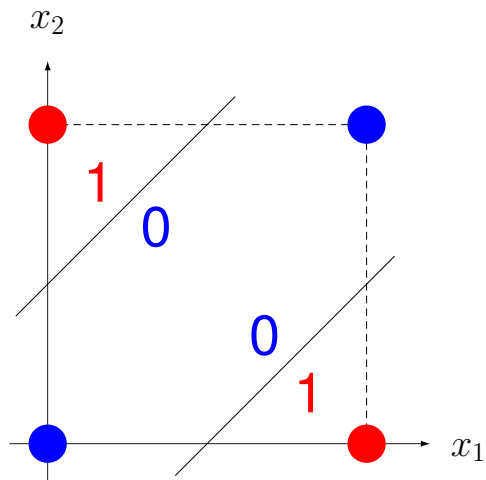
The XOR Problem



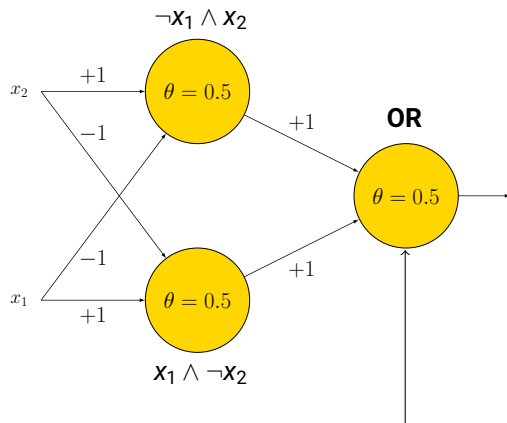
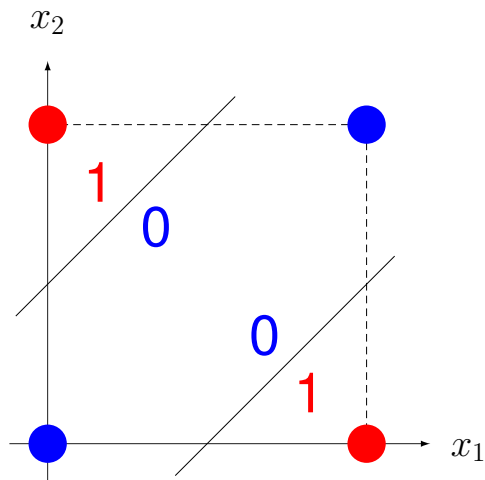
The XOR Problem



The XOR Problem



The XOR Problem

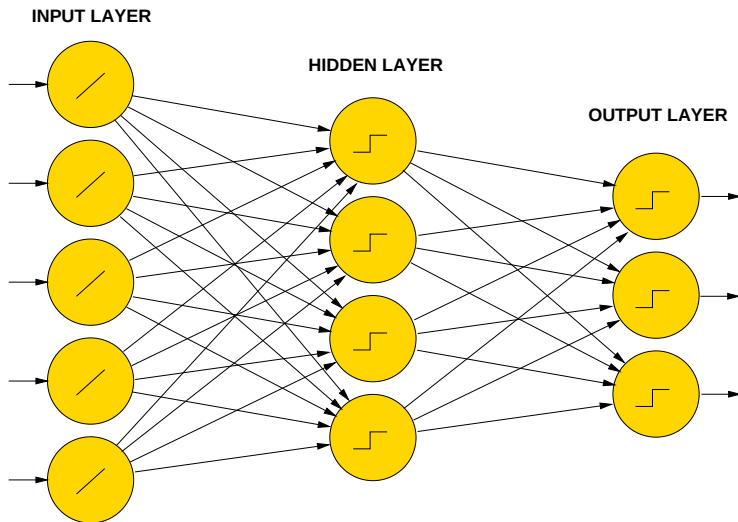


$$\text{XOR}(x_1, x_2) = (x_1 \wedge \neg x_2) \vee (\neg x_1 \wedge x_2)$$

Multi-Layer Perceptrons I

- The only solution to the limitation of linear separability is to introduce intermediate layers.
- A multi-layer perceptron is a feed-forward artificial neural network consisting of a certain number of layers of perceptrons.
- The output of such a network is computed in the following way: The outputs of the first layer are initialized with the input $(x_1, \dots, x_d)$, then the outputs of the other neurons are computed layer by layer using Formula (1).
- The “only problem” is how to find appropriate weights and thresholds that solve a given classification problem.

Multi-Layer Perceptrons II



Perceptron in History

- The perceptron learning algorithm is *not* applicable to multi-layer perceptrons, since we have no reference labels for the units in hidden layers.
- Minsky and Papert, the pioneers of perceptrons, conjectured in the late 1960ies that a training algorithm for multi-layer perceptrons --- even if one could be found --- is computationally infeasible and that, therefore, the study of multi-layer perceptrons is not worthwhile.
- Because of this conjecture, the study of multi-layer perceptrons was almost halted until the mid of the 1980ies.
- In 1986, Rumelhart and McClelland first published the *backpropagation algorithm* and, thereby, proved Minsky and Papert wrong.
- It turned out later that the backpropagation algorithm had already been described by Werbos in 1974 in his dissertation. In a different context, the algorithm first appeared in the work of Bryson *et al.* in the 1960ies.

Continuous Activation Functions

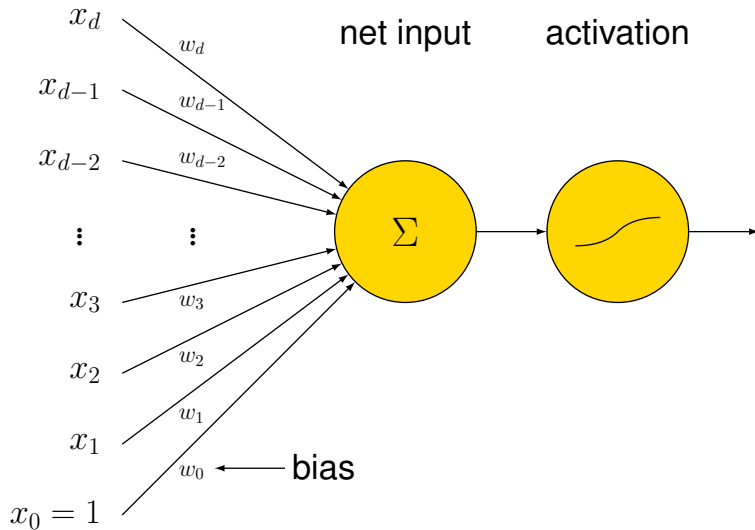
The key idea is to replace the discontinuous threshold function by a *differentiable function* φ . Then the output of the neuron, its so-called *activation*, is computed as

$$g(\vec{x}; \vec{w}, \theta) = \varphi\left(\sum_{j=1}^d w_j \cdot x_j - \theta\right). \quad (2)$$

For simplicity, we consider the offset $-\theta$ as a “zero-th” weight w_0 (with the convention $x_0 = 1$) and denote the activation as a and the so-called *network input* of the neuron as $\text{net} = \sum_{j=0}^d w_j \cdot x_j = w_0 + \sum_{j=1}^d w_j \cdot x_j$:

$$g(\vec{x}; \vec{w}) = a = \varphi\left(w_0 + \sum_{j=1}^d w_j \cdot x_j\right). \quad (3)$$

Unit With Continuous Activation



Examples of Continuous Activations

- For regression, the simplest choice is $\varphi(\mathbf{x}) = \mathbf{x}$ (i.e. resulting in a simple linear regressor).
- For 0/1 outputs, the most common choice is the so-called *sigmoid* or *logistic function*

$$\varphi(\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{x}}}.$$

Note that, in this particular case, $\varphi'(\mathbf{x}) = \varphi(\mathbf{x}) \cdot (1 - \varphi(\mathbf{x}))$ holds.

- For -1/+1 data, a common choice of the activation function is the *hyperbolic tangent*
 $\varphi(\mathbf{x}) = \tanh(\mathbf{x}) = \frac{e^{\mathbf{x}} - e^{-\mathbf{x}}}{e^{\mathbf{x}} + e^{-\mathbf{x}}}$, which is nothing else but a transformation of the sigmoid function to the interval $[-1, +1]$.

Training With Differentiable Activation Function

- The central problem with training a perceptron with threshold activation is the lack of differentiability. If a differentiable activation function and a differentiable loss function are used, we can compute the derivative of the loss of a single training sample (and, consequently, the empirical loss) according to the weights $\vec{w}$.
- The derivatives are the key to solving the learning problem, since an *iterative optimization method* can be used, e.g. *gradient descent*.

Logistic Regression and Cross Entropy I

Suppose we have a perceptron with sigmoid activation function φ . Then the output $g(\vec{x}^i; \vec{w})$ can be interpreted as an estimate of the probability that $\vec{x}^i$ belongs to the positive class:

$$p(y = 1 \mid \vec{x}^i) = \mathbf{a} = \varphi(\text{net}) \quad \text{and} \quad p(y = 0 \mid \vec{x}^i) = 1 - \mathbf{a} = 1 - \varphi(\text{net})$$

We can unify these two formulas as follows to get a single formula for the likelihood $p(y = y^i \mid \vec{x}^i; \vec{w})$:

$$\begin{aligned} p(y = y^i \mid \vec{x}^i; \vec{w}) &= \varphi(\text{net})^{y^i} \cdot (1 - \varphi(\text{net}))^{(1-y^i)} \\ &= \mathbf{a}^{y^i} \cdot (1 - \mathbf{a})^{(1-y^i)} \end{aligned}$$

Logistic Regression and Cross Entropy II

Maximizing the likelihood is equivalent to minimizing $-\mathbf{1}$ times its natural logarithm:

$$-\ln(p(y = y^i | \vec{x}^i; \vec{w})) = -y^i \cdot \ln(a) - (1 - y^i) \cdot \ln(1 - a)$$

This formula corresponds to the *cross entropy* between prediction and true class, and is a well-established loss function of its own.

Training a linear classifier by minimizing the cross entropy of a logistic linear model is usually called *logistic regression*.

Training Algorithm (Online Version)

1. Given: data set $\mathbf{Z} = \{(\vec{x}^i, y^i) \mid i = 1, \dots, l\}$, where $\vec{x}^i \in \mathbb{R}^d, y^i \in \mathbb{R}$; learning rate η ; initial weight vector $\vec{w}$
2. For all training samples $(\vec{x}^i, y^i)$ (in random order) do:
 - 2.1 Compute $\mathbf{net}$, $\mathbf{a}$, and δ
 - 2.2 Update: $\vec{w} := \vec{w} - \eta \cdot \delta \cdot \begin{pmatrix} 1 \\ \vec{x}^i \end{pmatrix}$
3. Return to 2. if stopping condition not fulfilled
4. Output: vector of weights $\mathbf{w} \in \mathbb{R}^{d+1}$

Training Algorithm (Batch Version)

1. Given: data set $\mathbf{Z} = \{(\vec{x}^i, y^i) \mid i = 1, \dots, l\}$, where $\vec{x}^i \in \mathbb{R}^d, y^i \in \mathbb{R}$; learning rate η ; initial weight vector $\vec{w}$
2. Set $\Delta\vec{w} = \vec{0}$
3. For all training samples $(\vec{x}^i, y^i)$ do:
 - 3.1 Compute net , $\mathbf{a}$, and δ
 - 3.2 $\Delta\vec{w} := \Delta\vec{w} - \eta \cdot \delta \cdot \begin{pmatrix} 1 \\ \vec{x}^i \end{pmatrix}$
4. Update: $\vec{w} := \vec{w} + \Delta\vec{w}$
5. Return to 2. if stopping condition not fulfilled
6. Output: vector of weights $\mathbf{w} \in \mathbb{R}^{d+1}$

Delta Term

1. If we use a sigmoid activation function along with the cross entropy loss (this combination corresponds to so-called *logistic regression*), the delta term is defined as (for a given sample $(\vec{x}^i, y^i)$)

$$\delta = a - y^i.$$

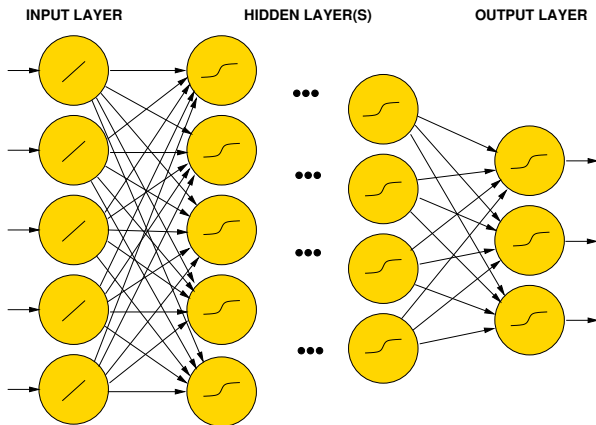
2. If we use the identity activation $\varphi(\mathbf{x}) = \mathbf{x}$ along with the quadratic loss (this combination corresponds to *classical linear regression*), the delta term is again given as (for a given sample $(\vec{x}^i, y^i)$)

$$\delta = a - y^i.$$

Training with Differentiable Activation

- One pass through the training set is usually called a *training epoch*.
- The first variant is called *online learning* because $\vec{w}$ is updated for each sample individually. This local update performs one gradient descent step with the aim to reduce the loss for this sample. The *randomization of the order of training samples* is necessary to avoid that the result is biased to the order of samples.
- The second variant is called *batch learning* because the local updates are summed up for all samples before $\vec{w}$ is updated. This global update performs one gradient descent step with the aim to reduce the empirical error for the entire training set.
- A common variant is to train in *mini batches*, i.e. to apply batch training for small randomly sampled batches. This is called *stochastic gradient descent (SGD)*.

Multi-Layer Perceptron: Architecture and Notation I



Multi-Layer Perceptron: Architecture and Notation II

Suppose that we have a training set with I real-valued inputs and O outputs (continuous or binary), i.e.

$$\mathbf{Z} = \{(\vec{x}^i, \vec{y}^i) \mid i = 1, \dots, I, \vec{x}^i \in \mathbb{R}^I, \vec{y}^i \in \mathbb{R}^O\}.$$

Then we consider a multi-layered network as follows:

- The network consists of an *input layer*, $N - 1$ *hidden/intermediate layer*, and one *output layer*.
- For each layer $k = 0, \dots, N$, we denote the number of neurons in that layer with n_k . The input layer has I neurons, i.e. $n_0 = I$ and the output layer has O neurons, i.e. $n_N = O$.
- The activation of each layer is denoted with $\vec{a}^{(k)}$.

Multi-Layer Perceptron: Architecture and Notation III

- For $k > 0$, each neuron of the k -th layer receives input from all neurons of the $k - 1$ -st layer and the *bias unit* (which has constant activation **1**), but not from any other neurons. The *net input* of the k -th layer is denoted with $\mathbf{net}^{(k)}$. The activation function used by all units of the k -th layer is denoted with φ_k .
- For each $k > 0$, the weights between the $k - 1$ -st and the k -th layer are stored in a *weight matrix* $\mathbf{W}^{(k)} = (w_{ji}^{(k)})_{j=1, \dots, n_k}^{i=0, \dots, n_{k-1}}$. So, $w_{ji}^{(k)}$ is the weight of the connection between the i -th unit of the $k - 1$ -st layer and the j -th unit of the k -th layer. Any $w_{j0}^{(k)}$ corresponds to the weight between the bias unit and the j -th unit of the k -th layer.
- Networks in which all units of one layer are connected with all units of the next layer are called *fully connected neural networks*.

Forward Propagation / Forward Pass

1. Given: multi-layer perceptron according to the specifications above, i.e. with I inputs, O outputs and $N - 1$ hidden layers; input $\vec{x} \in \mathbb{R}^I$.
2. Set $\vec{a}^{(0)} := \vec{x}$.
3. For k from 1 to N do:
 - 3.1 Compute net input as $\mathbf{net}^{(k)} := \mathbf{W}^{(k)} \cdot \left(\vec{a}^{(k-1)} \right)$.
 - 3.2 Compute activation as $\vec{a}^{(k)} := \varphi_{k\bullet}(\mathbf{net}^{(k)})$ (apply φ_k element-wise to $\mathbf{net}^{(k)}$).
4. Output: $\vec{y} := \vec{a}^{(N)}$.

So the input $\vec{x}$ is *propagated* through the network from the input layer to the output layer. As already noted, such a type of network is called *feed-forward network*.

Backpropagation Algorithm (Online Version)

1. Given: data set $\mathbf{Z} = \{(\vec{x}^i, \vec{y}^i) \mid i = 1, \dots, I\}$, where $\vec{x}^i \in \mathbb{R}^I$, $\vec{y}^i \in \mathbb{R}^O$; learning rate η ; network as above with some choice of initial weights.
2. For all training samples $(\vec{x}^i, \vec{y}^i)$ (in random order) do:
 - 2.1 Propagate $\vec{x}^i$ through network to compute all network inputs and activations.
 - 2.2 Compute $\delta^{(N)}$.
 - 2.3 Set $\Delta \mathbf{W}^{(N)} := -\eta \cdot \delta^{(N)} \cdot (\mathbf{1} \mid \vec{a}^{(N-1)T})$.
 - 2.4 For all k from $N - 1$ to 1 do:
 - 2.4.1 Compute $\delta^{(k)} = \text{diag}(\varphi'_{k\bullet}(\text{net}^{(k)})) \cdot \mathbf{W}^{(k+1)T} \cdot \delta^{(k+1)}$.
 - 2.4.2 Set $\Delta \mathbf{W}^{(k)} := -\eta \cdot \delta^{(k)} \cdot (\mathbf{1} \mid \vec{a}^{(k-1)T})$.
 - 2.5 Update: For all k from 1 to N do
 - 2.5.1 $\mathbf{W}^{(k)} := \mathbf{W}^{(k)} + \Delta \mathbf{W}^{(k)}$.
3. Return to 2. if stopping condition not fulfilled
4. Output: weights $\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(N)}$.

Backpropagation Algorithm (Batch Version)

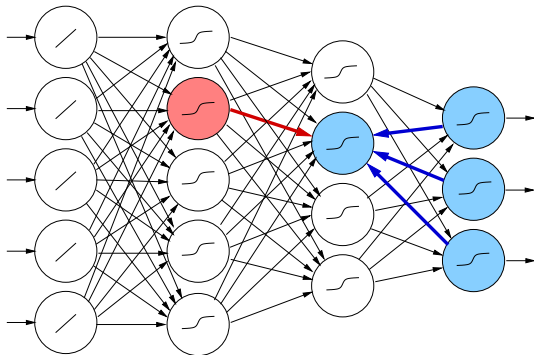
1. Given: data set $\mathbf{Z} = \{(\vec{x}^i, \vec{y}^i) \mid i = 1, \dots, I\}$, where $\vec{x}^i \in \mathbb{R}^I$, $\vec{y}^i \in \mathbb{R}^O$; learning rate η ; network as above with some choice of initial weights.
2. Set $\Delta \mathbf{W}^{(1)}, \dots, \Delta \mathbf{W}^{(N)}$ to zero matrices.
3. For all training samples $(\vec{x}^i, \vec{y}^i)$ do:
 - 3.1 Propagate $\vec{x}^i$ through network to compute all network inputs and activations.
 - 3.2 Compute $\delta^{(N)}$.
 - 3.3 Set $\Delta \mathbf{W}^{(N)} := \Delta \mathbf{W}^{(N)} - \eta \cdot \delta^{(N)} \cdot (1 \mid \vec{a}^{(N-1)T})$.
 - 3.4 For all k from $N - 1$ to 1 do:
 - 3.4.1 Compute $\delta^{(k)} = \text{diag}(\varphi'_{k\bullet}(\text{net}^{(k)})) \cdot \mathbf{W}^{(k+1)T} \cdot \delta^{(k+1)}$.
 - 3.4.2 Set $\Delta \mathbf{W}^{(k)} := \Delta \mathbf{W}^{(k)} - \eta \cdot \delta^{(k)} \cdot (1 \mid \vec{a}^{(k-1)T})$.
4. Update: For all k from 1 to N do
 - 4.1 $\mathbf{W}^{(k)} := \mathbf{W}^{(k)} + \Delta \mathbf{W}^{(k)}$.
5. Return to 2. if stopping condition not fulfilled
6. Output: weights $\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(N)}$.

Backpropagation Algorithm

- One pass through the training set is called a *training epoch*.
- The first variant is called *online learning* because the weights are updated for each sample individually. This local update performs one gradient descent step with the aim to reduce the loss for this sample. The *randomization of the order of training samples* is necessary to avoid that the result is biased to the order of samples.
- The second variant is called *batch learning* because the local updates are summed up for all samples before weights are updated. This global update performs one gradient descent step with the aim to reduce the empirical error for the entire training set.
- As mentioned before, training in *mini batches* is common, too.
- Online learning and mini batches do not consider the exact gradient, but an approximation that depends on the random choice of samples. This is called *stochastic gradient descent (SGD)*.

Delta Term

- The deltas in the last layer are computed as above.
- The deltas of the $N - 1$ -st layer are given as the derivative of the activation function times a weighted sum of deltas of the N -th layer, i.e. the deltas are *propagated back* through the network.
- In the same way, the deltas are propagated back to the $N - 2$ -nd layer and so forth.



Derivative of Loss Function

With the above assumptions, the derivative of the loss function w.r.t. to weight $w_{ji}^{(N-1)}$, i.e. a weight between the $N - 2$ -nd and the $N - 1$ -st layer, is given as:

$$\begin{aligned}
 \frac{\partial L}{\partial w_{ji}^{(N-1)}}(\vec{y}, \vec{a}^{(N)}) &= \sum_{h=1}^{n_N} \frac{\partial L}{\partial a_h^{(N)}}(\vec{y}, \vec{a}^{(N)}) \cdot \overbrace{\frac{\partial a_h^{(N)}}{\partial a_j^{(N-1)}}(a_j^{(N-1)})}^{=\varphi'_N(\text{net}_h^{(N)}) \cdot w_{hj}^{(N)}} \cdot \overbrace{\frac{\partial a_j^{(N-1)}}{\partial w_{ji}^{(N-1)}}(a_i^{(N-1)}, w_{ji}^{(N-1)})}^{=\varphi'_{N-1}(\text{net}_j^{(N-1)}) \cdot a_i^{(N-2)}} \\
 &= \underbrace{\varphi'_{N-1}(\text{net}_j^{(N-1)}) \cdot \sum_{h=1}^{n_{N-1}} \overbrace{\varphi'_N(\text{net}_h^{(N)}) \cdot \frac{\partial L}{\partial a_h^{(N)}}(\vec{y}, \vec{a}^{(N)}) \cdot w_{hj}^{(N)} \cdot a_i^{(N-2)}}^{=\delta_h^{(N)}}}_{=\delta_j^{(N-1)}} \\
 &= \underbrace{\varphi'_{N-1}(\text{net}_j^{(N-1)}) \cdot \left(\sum_{h=1}^{n_{N-1}} \delta_h^{(N)} \cdot w_{hj}^{(N)} \right)}_{=\delta_j^{(N-1)}} \cdot a_i^{(N-2)}
 \end{aligned} \tag{4}$$

Backpropagation Algorithm - Summary

Forward pass: Set activation of input layer to input vector. For each layer (from first hidden layer to output layer), compute net input as product of activation with weight matrix and apply activation function to net input for each unit.

Backward pass: Perform forward pass and compute deltas for output layer. For each layer (from last hidden layer to first hidden layer), compute deltas as elementwise products of the activations' derivatives and the product of the transpose of the weight matrix times the deltas of the next layer (the one "to the right"). The weight updates are outer products of the deltas of the "right layer" and the activations of the "left layer".

Multi-Layer Perceptron - Classification I

- Assume we are given a data set $\mathbf{Z} = \{(\vec{x}^i, y^i) \mid i = 1, \dots, l\}$.
- If we have a binary classification problem, i.e. $y_i \in \{0, 1\}$ or $y_i \in \{-1, +1\}$ (in this case, replace -1 's by 0 's), we can use a single output neuron ($O = 1$).
- If we are given a problem with $M > 2$ classes, i.e. $y^j \in \{1, \dots, M\}$, we can use a network with $O = M$ output neurons. The labels have to be mapped to M -dimensional output vectors $\vec{y}^j$ in the following way (*one-hot encoding*):

$$y_j^i = \begin{cases} 1 & \text{if } y^i = j \\ 0 & \text{otherwise} \end{cases}$$

When predicting a new sample $\vec{x}$, the output neuron with the highest activation determines the class prediction.

Multi-Layer Perceptron - Classification II

- If we use sigmoid activation functions for multi-class problems with more than 2 output neurons, the activations need not sum up to 1, therefore, they cannot be used as estimates of class probabilities.
- For multi-class classification, it is more common to use the *softmax* activation function:

$$a_i^{(N)} = \frac{\exp(\text{net}_i^{(N)})}{\sum_{j=1}^O \exp(\text{net}_j^{(N)})}$$

- If we use the softmax activation function in conjunction with the multi-class cross entropy loss

$$L(\vec{y}, \vec{a}^{(N)}) = - \sum_{i=1}^O y_i \cdot \ln(a_i^{(N)}),$$

the deltas of the output layer again simplify to $\delta^{(N)} = (\vec{a}^{(N)} - \vec{y})$.

Multi-Layer Perceptron - Regression

- Assume we are given a data set $\mathbf{Z} = \{(\vec{x}^i, \vec{y}^i) \mid i = 1, \dots, l\}$, where $\vec{x}^i \in \mathbb{R}^l$ and $\vec{y}^i \in \mathbb{R}^o$ (in the simplest case $\mathbf{O} = \mathbf{1}$)
- There are two ways to make multi-layer perceptrons usable for regression:
 1. Transforming/scaling all desired output vectors $\vec{y}^i$ to $[0, 1]^o$
 2. The better option is to use linear neurons in the output layer, i.e., $\varphi_N(\mathbf{x}) = \mathbf{x}$ is used, while the other neurons remain unchanged; in this case, the outputs of the $\mathbf{N} - \mathbf{1}$ -st layer can be understood as basis functions; the output is then a linear combination of these basis functions.
- The most common loss function is the quadratic loss.
- Multi-layer perceptrons are universal approximators, however, this is only a theoretical result with minor practical value.

Usage Scenario: Autoencoders

- Assume we are given a data set $\mathbf{X} = \{\vec{x}^i \mid i = 1, \dots, l\}$, where $\vec{x}^i \in \mathbb{R}^d$. (only inputs, no targets!)
- Now consider a multi-layer perceptron with $l = d$ input neurons and $o = d$ output neurons and suppose that we train the network with the training set $\mathbf{Z} = \{(\vec{x}^i, \vec{x}^i) \mid i = 1, \dots, l\}$, i.e. the network is trained to produce its input as output. Such a network is called *autoencoder*.
- If the number of units in each hidden layer is smaller than d or if other precautions against overfitting are taken (see later), then such an autoencoder learns an efficient representation of the data set.
- It is also possible to train an autoencoder on a data set whose input vectors have been perturbed by noise. Then the autoencoder learns to recover original data from noisy data. Such a network is commonly called *denoising autoencoder*.

Practical Considerations I

Input scaling: It is advisable to standardize all inputs to the same range, e.g. to $[-1, 1]$ or to mean 0 and variance 1. This ensures that all input features have the same prior importance. Otherwise large input features are overrated and it might take long to properly adjust the weights for small input features.

Initial weights: A common strategy is to use small random values, e.g. drawn from a uniform distribution over $[-0.1, 0.1]$. Weights around 0 result in net inputs around 0. Therefore, the network is almost linear in the beginning. Moreover, the derivatives of the sigmoid activation function is maximal around 0, which speeds up learning in the beginning.

Number of hidden layers: For most learning tasks, two hidden layers are sufficient. For simpler tasks, even one or no hidden layer may be sufficient. Larger numbers of hidden layers would make sense for more difficult tasks, but appropriate precautions have to be taken against overfitting. Moreover, training becomes increasingly difficult for deeper networks (see later).

Practical Considerations II

Numbers of hidden units: Too few hidden units result in underfitting. Too many hidden units may result in overfitting unless appropriate precautions, such as, regularization are taken (see later). Moreover, pruning or growing strategies can be applied during training.

Learning rates: Typically, learning rates have to be low for online learning, e.g. 0.1 or 0.01. For batch training, it is common to use something like, e.g., $\eta = \frac{0.7}{I}$. The best choice, however, is to adapt the learning rate during training. This can be done by help of the Hessian (advanced topic, see literature).

Online vs. batch: With an appropriately chosen learning rate (or an adaptive learning rate), online learning is known to converge faster than batch learning (see literature). It is also possible to choose a strategy in between: by training in *mini-batches*.

Practical Considerations III

Momentum term: In order to avoid oscillations, it is possible to augment updates with the previous update, i.e. $\vec{w} := \vec{w} + \Delta\vec{w} + \mu \cdot \Delta\vec{w}^{\text{old}}$, where $\Delta\vec{w}^{\text{old}}$ is the weight update of the previous iteration/epoch. μ is the momentum parameter that controls the influence of the previous update.

Stopping criteria: The following stopping criteria can be used (possibly in combination):

- Maximal number of epochs/iterations reached
- Empirical error below certain threshold
- Relative improvement of empirical below certain threshold
- Maximal weight change below certain threshold

Regularization I

- Support vector machines include a capacity term in its objective: the flatness of discriminant/regressor function (which corresponds to margin maximization for classification). In theory, this capacity term aims to avoid overfitting by limiting model complexity.
- The neural networks considered so far have no such built-in mechanism against overfitting.
- In neural networks, complexity is mainly controlled by the architecture of the network (number of hidden layers along with numbers of neurons in hidden layers).
- Since it is hard to find the optimal architecture for a given learning task, an option would be to allow for a larger (more complex) network, but to control its model complexity by appropriate additional measures during training.

Regularization II

Early stopping: As mentioned above, if we start from initial weights around 0, the neurons' activation is nearly linear. So the model complexity is low in the beginning. During the course of training, weights will become larger and the neurons' activation becomes more non-linear, i.e. the model complexity grows. The idea of early stopping is to quit learning when the right model complexity is reached. There are no well-founded strategies for that. Instead, the use of a validation set is more or less the only option.

Training with noise: In order to avoid that the networks adapts too much to individual training samples (overfitting), noise is added either to the inputs or to the weights during training. Different strategies are available for that (see literature).

Regularization III

Weight decay: As mentioned above, weights around 0 correspond to low complexity models. A higher model complexity necessitates higher weights (in terms of their absolute values). Therefore, a mechanism that favors weights around 0 can be used to control model complexity. In other words, we add $\lambda \cdot \Omega(\mathcal{W})$ to the learning objective, where the *regularization term* $\Omega(\mathcal{W})$ measures the overall size of the weights, $\mathcal{W}$ is the set of all weights (resp. weight matrices) in the network and λ is the *regularization parameter* that controls the influence of the regularization term. A basic variant is *Tikhonov regularization* in which $\Omega(\mathcal{W})$ is the (halved) *sum of squares of all weights*. This results in a derivative of $\frac{\partial \Omega}{\partial \mathbf{w}_{ji}^{(k)}}(\mathcal{W}) = \mathbf{w}_{ji}^{(k)}$ resp. $\frac{\partial \Omega}{\partial \mathbf{W}^{(k)}}(\mathcal{W}) = \mathbf{W}^{(k)}$. Hence, the update in backpropagation changes to

$$\mathbf{W}^{(k)} := \mathbf{W}^{(k)} + \Delta \mathbf{W}^{(k)} - \lambda \cdot \mathbf{W}^{(k)},$$

where $-\lambda \cdot \mathbf{W}^{(k)}$ obviously tends to push the weight matrix towards **0**.

Regularization IV

Alternative Learning Algorithms

As backpropagation with fixed learning rate often converges slowly, add-ons and alternatives have been established:

Adaptive learning rate: The learning rate is not constant, but varied according to some strategy/heuristics.

Resilient backpropagation (Rprop): only the signs of gradients are taken into account to determine the direction of the weight update, where learning rates are adapted for each weight individually.

Root Mean Square Propagation (RMSProp): learning rates are adapted for each weight individually by dividing the learning rate by a running average of the magnitudes of recent gradients for that weight.

Adaptive Moment Estimation (Adam): extension of RMSProp that also takes the second moments of the gradients into account.

Second-order methods: methods that not only use first derivatives, but also second-order derivatives (Newton method) or approximations of second-order derivatives (quasi-Newton methods, BFGS, Levenberg-Marquardt).

Artificial Neural Networks

- + Universal
- + Relatively easy to apply
- Black box
- Only applicable for vectorial data
- Large effort for training
- Solution is not guaranteed to be a global minimum, but only a local one
- Large number of different variants and parameters; thorough parameter selection is not feasible because of large number of parameters and long training times.

Deep Networks: Motivation

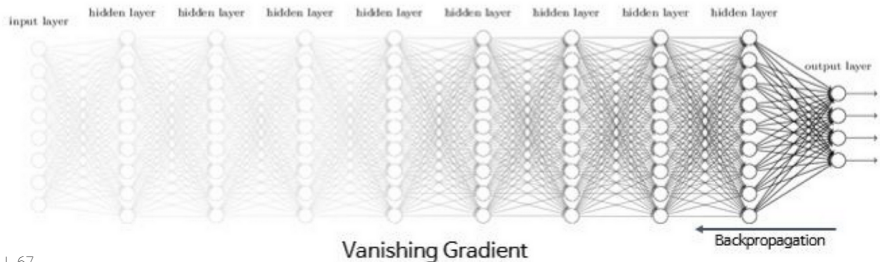
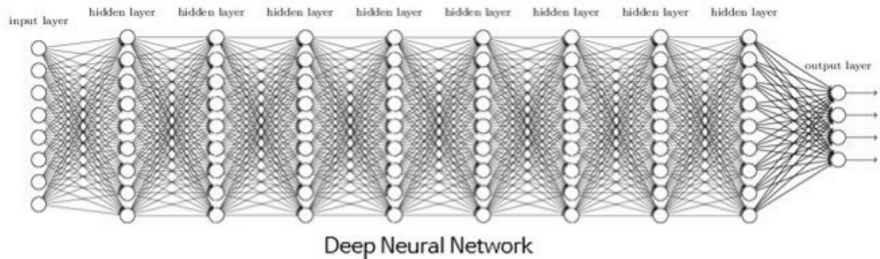
- Neural networks that are deep (many hidden layers) and broad (many neurons in each hidden layers) would allow for representing, storing, and recognizing many complex patterns.
- However, there are practical obstacles to the application of standard backpropagation (and similar variants) to such networks:
 - Overfitting is highly likely to occur (which could possibly be counteracted with some regularization).
 - Backpropagation cannot be used for training more than 2--3 hidden layers because of *vanishing gradients* (see next page).

The Vanishing Gradient Problem

$$\begin{aligned}\frac{\partial L}{\partial \mathbf{W}^{(k-1)}}(\vec{y}, \vec{a}^{(N)}) &= \delta^{(k-1)} \cdot (1 \mid \vec{a}^{(k-2)^T}) \\ &= \text{diag}(\varphi'_{k-1\bullet}(\text{net}^{(k-1)})) \cdot \mathbf{W}^{(k)^T} \cdot \delta^{(k)} \cdot (1 \mid \vec{a}^{(k-2)^T}),\end{aligned}$$

i.e. each backward propagation step includes a multiplication with derivatives of activation functions. If we have back-propagated deltas from the N -th layer to the $N - q$ -th layer, we have performed such multiplications q times. Since the maximal value of the derivative of the sigmoid/logistic function is $\frac{1}{4}$, this means that the magnitude of the deltas (and therefore the magnitude of the gradients and the updates) decreases exponentially layer by layer. This makes backpropagation training of deep networks intractable, as weights at the back are updated extremely slowly.

The Vanishing Gradient Problem



Deep Learning

- *Deep learning* is a class/framework of strategies for training deep networks that are aimed to learn multiple levels of representations of the data and to allow for accurate predictions from these representations.
- Deep Learning rests on the following three pillars:
 1. Availability of new network architectures and new methods for training large/deep networks
 2. Availability of big data sets
 3. Availability of resources for training large/deep networks (in particular, Graphical Processing Units --- GPUs)
- Deep Learning most often suffices without explicit feature engineering --- complex representations/features are automatically learned from big data.

First Approaches to Deep Learning

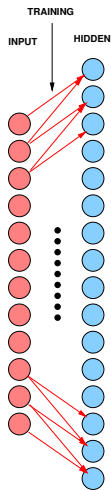
In the first successful supervised applications, deep learning was a two-step procedure:

Pre-training: levels of representations are learned layer by layer (unsupervised or supervised).

Fine-tuning: a supervised learning algorithm is applied that makes predictions from the activations of the last layer of the pre-trained network. Typically a single-layer ANN is used, but any other predictor would be possible too.

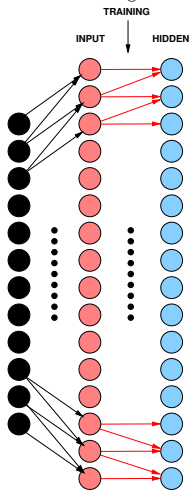
Pre-Training and Fine-Tuning

Pre-training: hidden layer no. 1



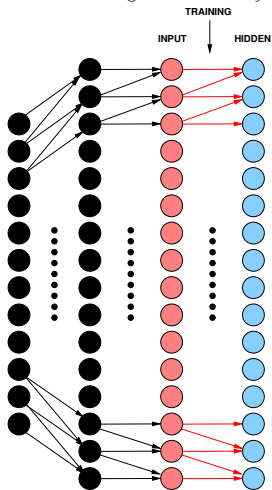
Pre-Training and Fine-Tuning

Pre-training: hidden layer no. 2



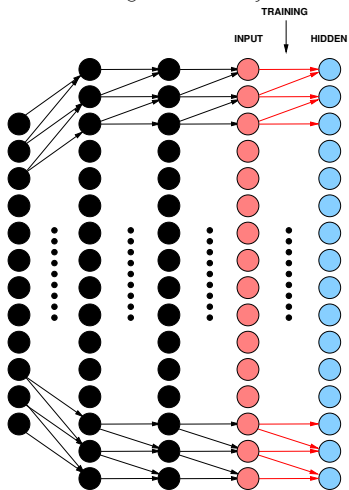
Pre-Training and Fine-Tuning

Pre-training: hidden layer no. 3



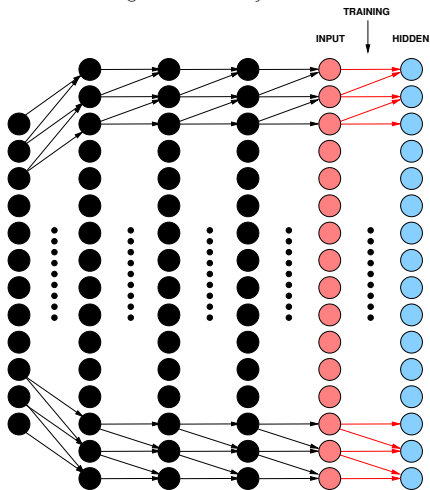
Pre-Training and Fine-Tuning

Pre-training: hidden layer no. 4



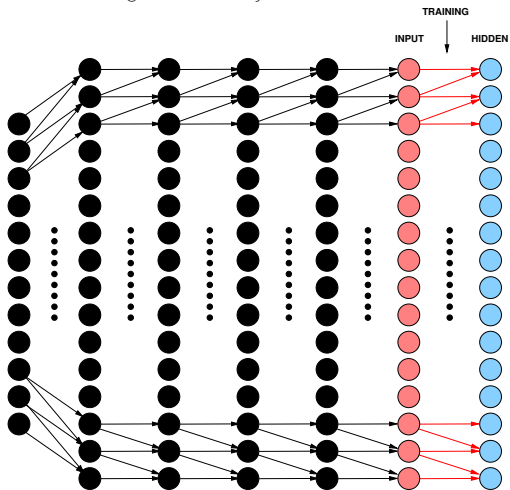
Pre-Training and Fine-Tuning

Pre-training: hidden layer no. 5



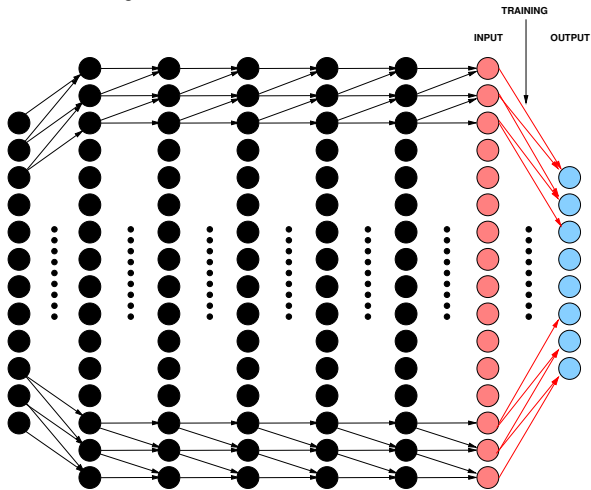
Pre-Training and Fine-Tuning

Pre-training: hidden layer no. 6



Pre-Training and Fine-Tuning

Fine-tuning:



Methods for Pre-Training

Restricted Boltzmann machine (RBM): A simple stochastic neural network with an input layer and one hidden layer that are connected in both directions with symmetric weights; RBMs aim to learn a probability distribution over the inputs. The learning algorithm uses sampling of inputs and hidden activations along with gradient descent.

Autoencoders: A (denoising) autoencoder with one hidden layer is trained in each pre-training step. After training, the output layer of the autoencoder is discarded and only the hidden layer remains. In the subsequent step, another autoencoder is trained with the inputs being the activations of the hidden neurons of the previously trained autoencoder.

Supervised pre-training: A network with one hidden layer is trained in each pre-training step. After training, the output layer is discarded and only the hidden layer remains. In the subsequent step, another network is trained with the inputs being the activations of the hidden neurons of the previous network.

Deep Learning (cont'd)

- Unsupervised deep learning only consists of unsupervised pre-training and omits fine-tuning.
- In the meantime, it has become common to apply *deep learning in a purely supervised fashion*, i.e. without pre-training. In order to avoid the vanishing gradient problem and to facilitate meaningful data representations, appropriate regularization measures need to be employed (see next slides).

How to Learn Good Representations?

- The success of a deep network is determined by how meaningful the representations in the hidden layers are.
- What is a meaningful representation?
 - Each hidden unit corresponds to a specific pattern (hidden) in the data.
 - Each hidden layer corresponds to a certain *level of abstraction*.
 - Different hidden units correspond to different patterns, i.e. the patterns are disentangled. Otherwise, further representations are difficult to learn and/or the final supervised learning in the fine-tuning step is difficult.

In the following, we highlight a few approaches aimed at meaningful representations.

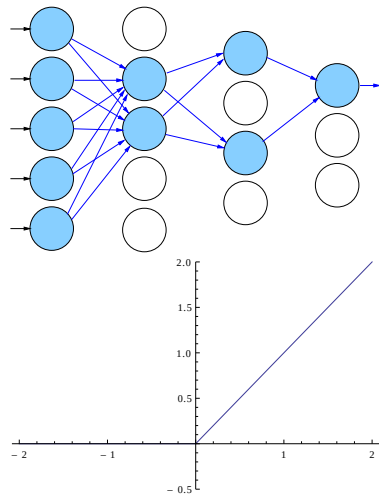
Learning Sparse Representations

Disentangling of representations can also be achieved by ensuring sparse activation, i.e. only a fraction of hidden neurons are activated for a given input.

Dropout: during training, activations are randomly set to 0 (typically with a probability of 0.5);

Rectified linear units (ReLU): instead of a sigmoid activation function, a function is used that gives 0 below a certain threshold. The most common choice is $\varphi(\mathbf{x}) = \max(0, \mathbf{x})$.

These approaches allow for training a deep network directly without pre-training.



Other Activation Functions I

Leaky ReLU:

$$\varphi(\mathbf{x}) = \begin{cases} \mathbf{x} & \text{if } \mathbf{x} > 0 \\ \alpha \cdot \mathbf{x} & \text{otherwise} \end{cases}$$

Exponential Linear Unit (ELU):

$$\varphi(\mathbf{x}) = \begin{cases} \mathbf{x} & \text{if } \mathbf{x} > 0 \\ \mathbf{e}^{\mathbf{x}} - 1 & \text{otherwise} \end{cases}$$

Other Activation Functions II

Scaled ELU:

$$\varphi(\mathbf{x}) = \lambda \cdot \begin{cases} \mathbf{x} & \text{if } \mathbf{x} > 0 \\ \alpha \cdot (\mathbf{e}^{\mathbf{x}} - 1) & \text{otherwise} \end{cases}$$

The goal of this function is to ensure that activations have a mean of zero and variance of 1 throughout training. Used along with proper initialization, this can be ensured with $\alpha = \mathbf{1.6732}$ and $\lambda = \mathbf{1.0507}$. Networks using Scaled ELUs are called *self-normalizing networks*, as they do no longer require any normalization to ensure even gradient flow.

Deep Learning: Main Applications

- Not very common, but also successful: applying deep learning concepts to fully connected networks (vectorial data)
- Computer vision: convolutional neural networks
- Sequences, speech and language processing: recurrent neural networks
- Generation of artificial data: generative adversarial networks
- Many combinations are possible, e.g. convolutional recurrent networks for video analysis, image-to-speech captions, etc.

Example: Tox21 Challenge I

- Computational challenge set up by the US agencies NIH, EPA, and FDA
- Unprecedented multi-million-dollar effort
- 12,000 compounds tested experimentally for twelve different toxic effects
- Goal: predict toxicity computationally

Example: Tox21 Challenge II

Input features:

- 40,000 very sparse features: Extended Connectivity FingerPrint (ECFP4) presence count of chemical sub-structures
- 5,057 additional features:
 - 2,500 toxicophore features
 - 200 common chemical scaffolds
 - various chemical descriptors

Example: Tox21 Challenge III

- Deep learning-based solution by JKU's Institute of Bioinformatics (now Institute for Machine Learning) won the grand challenge, both panels (nuclear receptor panel and stress response panel), and six single prediction tasks.
- The hierarchical representation of deep networks allowed for the identification of novel toxicophores.

